

Supplementary Material: Hyperledger Fabric Chaincode Algorithms (Updated for cardamom_11)

This supplementary material provides the algorithmic descriptions of the core functions used in the updated Hyperledger Fabric chaincode (`cardamom_11.js`). Long algorithms have been split into multiple parts, and long lines have been wrapped to fit within the page margins.

Algorithm 1: depositMoney()

Input: org, userId, amount

Output: Updated ledger state

```
1 this._requireOrg(ctx, 'BankMSP') // caller must be bank;
2 txId = ctx.stub.getTxID();
3 ts = this._simNowISO(ctx);
4 amt = Number(amount);
5 if (!Number.isFinite(amt) — amt != 0) {;
6   Abort transaction ('Invalid deposit amount: ${amount}');
7 end;
8 walletKey = '${org}-${userId}-wallet';
9 walletBytes = Fetch walletKey from ledger;
10 if (!walletBytes — walletBytes.length === 0) {;
11   Abort transaction ('Wallet not found for ${org}-${userId}');
12 end;
13 wallet = Parse JSON walletBytes.toString();
14 currentCount = wallet.consolidationCount — 0;
15 objectType = 'walletTx_${currentCount}';
16 key = ctx.stub.createCompositeKey(objectType, [org, userId, txId]);
17 wid = this._walletId(org, userId);
18 entry = {;
19   docType: 'walletTx';;
20   walletId: wid;;
21   org, userId;;
22   kind: 'DEPOSIT';;
23   delta: amt, // signed amount (+);
24   ts, txId;
25 end;
26 Update ledger with key, Buffer.from(Serialize entry);
27 return JSON.stringify({;
28   ok: true;;
29   txId;;
30   walletId: wid;;
31   amount: amt;;
32   balanceComputed: false;;
33   ts;
34 });;
```

Algorithm 2: transferMoney()

Input: fromOrg, fromUserId, toOrg, toUserId, amount

Output: Updated ledger state

```
1 return Transfer tokens ctx, fromOrg, fromUserId, toOrg, toUserId,
2   amount;
```

Algorithm 3: submitProduce() (Part 1 of 2)

Input: lotId, farmerId, weightKg, lotDate, bags, auctioncenterId

```
1 this.requireOrg(ctx, 'FarmersMSP');
2 if (!lotId —— !farmerId —— !weightKg —— !lotDate —— !bags ——
3   !auctioncenterId) {;
4   Abort transaction ('lotId, farmerId, weightKg, lotDate, bags,
5     auctioncenterId are required');
6   end;
7   weightNum = Number(weightKg);
8   bagsInt = parseInt(bags, 10);
9   if (!Number.isFinite(weightNum) —— weightNum != 0) Abort
      transaction
10    ('Invalid weightKg: ${weightKg}');
11   if (!Number.isInteger(bagsInt) —— bagsInt != 0) Abort transaction
12    ('Invalid bags: ${bags}');
13   feeKey = ctx.stub.createCompositeKey('testingFee',
14     [String(auctioncenterId)]);
15   feeData = Fetch feeKey from ledger;
16   if (!feeData?.length) {;
17     Abort transaction ('Auctioncenter fee not set for ID
18       ${auctioncenterId}');
19   end;
20   feeAmt;
21   try {;
22     feeObj = Parse JSON feeData.toString();
23     feeAmt = Number(feeObj.feeAmount);
24     if (!Number.isFinite(feeAmt) —— feeAmt != 0) {;
25       Abort transaction ('feeAmount must be a non-negative number (got
26         ${feeObj.feeAmount})');
27     end;
28   } catch (err) {;
29     Abort transaction ('Failed to parse testing fee for
30       ${auctioncenterId}: ${err.message}');
31   end;
32   Transfer tokens ctx, 'farmers', String(farmerId, 'auctioncenters',
33     String(auctioncenterId), String(feeAmt));
34   lot = {;
35     docType: 'lot';;
36     lotId: String(lotId);;
37     farmerId: String(farmerId);;
38     ownerId: String(farmerId);;
39     weightKg: weightNum;;
40     lotDate: String(lotDate);;
```

Algorithm 4: submitProduce() (Part 2 of 2)

Output: Updated ledger state

```
1 bags: bagsInt;;
2 auctioncenterId: String(auctioncenterId);;
3 status: 'SUBMITTED';;
4 submittedAt: this._simNowISO ? this._simNowISO(ctx) : new
5   Date().toISOString();;
6 testingFee: feeAmt;;
7 offers: [];
8 end;
9 lotKey = ctx.stub.createCompositeKey('lot', [String(lotId)]);
10 Update ledger with lotKey, Buffer.from(Serialize lot);
11 return Serialize { ok: true, lotId: String(lotId), testingFee: feeAmt
12   };
```

Algorithm 5: testCardamom() (Part 1 of 2)

Input: lotId, result, videoHash, gradingJson

```
1 this.requireOrg(ctx, 'AuctioncentersMSP');
2 lotKey = ctx.stub.createCompositeKey('lot', [lotId]);
3 lotBytes = Fetch lotKey from ledger;
4 if (!lotBytes — lotBytes.length === 0) Abort transaction ('Lot not
5   found');
6 lot = Parse JSON lotBytes.toString();
7 lot.status = result === 'pass' ? 'APPROVED' : 'REJECTED';
8 lot.testResult = result;
9 lot.testedAt = this.simNowISO(ctx);
10 lot.videoHash = videoHash;
11 if (gradingJson) {;
12   grading = Parse JSON gradingJson;
13   if (!grading.sizeGrades — typeof grading.sizeGrades !== 'object') {;
14     Abort transaction ("Missing or invalid 'sizeGrades' field in grading
15       data");
16   end;
17   for (const [size, breakdown] of Object.entries(grading.sizeGrades)) {;
18     { clean, sick, split, total } = breakdown;
19     if (;
20       clean === undefined —;
21       sick === undefined —;
22       split === undefined —;
23       total === undefined;
24     ) {;
25       Abort transaction ('Missing grading fields for size category:
26         ${size}');
27     end;
28   end;
29   requiredQuality = ['greenPercent', 'averagePercent', 'fruitPercent',
30     'belowAveragePercent'];
31   requiredMetrics = ['literWeight', 'moisture', 'numberOfBags',
32     'netWeight'];
33   for (const field of requiredQuality) {;
34     if (grading[field] === undefined) {;
35       Abort transaction ('Missing quality field: ${field}');
36     end;
37   end;
38   for (const field of requiredMetrics) {;
39     if (grading[field] === undefined) {;
40       Abort transaction ('Missing metric field: ${field}');
```

Algorithm 6: testCardamom() (Part 2 of 2)

Output: Updated ledger state

```
1 end;
2 end;
3 lot.grading = {;
4 sizeGrades: grading.sizeGrades;;
5 quality: {;
6 greenPercent: grading.greenPercent;;
7 averagePercent: grading.averagePercent;;
8 fruitPercent: grading.fruitPercent;;
9 belowAveragePercent: grading.belowAveragePercent;;
10 };;
11 metrics: {;
12 literWeight: grading.literWeight;;
13 moisture: grading.moisture;;
14 numberOfBags: grading.numberOfBags;;
15 netWeight: grading.netWeight;;
16 end;
17 end;
18 end;
19 Update ledger with lotKey, Buffer.from(Serialize lot);
20 return 'Lot ${lotId} tested as ${lot.status}${gradingJson ? '
21   with grading details' : ''}';
```

Algorithm 7: openAuction() (Part 1 of 2)

Input: lotId, reservePricePerKg, minIncrementPerKg

```
1 this.requireOrg(ctx, 'AuctioncentersMSP');
2 lotKey = ctx.stub.createCompositeKey('lot', [lotId]);
3 lotBytes = Fetch lotKey from ledger;
4 if (!lotBytes?.length) Abort transaction ('Lot not found');
5 lot = Parse JSON lotBytes.toString();
6 if (lot.status !== 'APPROVED') {;
7   Abort transaction ('Lot ${lotId} not eligible for auction
8     (status=${lot.status}) must be APPROVED');
9   end;
10 reserve =;
11 reservePricePerKg === " —— reservePricePerKg === null ——
12   reservePricePerKg === undefined;
13 ? null;
14 : Number(reservePricePerKg);
15 if (reserve !== null && (!Number.isFinite(reserve) —— reserve ; 0))
16   {;
17   Abort transaction ('Invalid reservePricePerKg');
18   end;
19 inc = Number(minIncrementPerKg);
20 if (!Number.isFinite(inc) —— inc ;= 0) {;
21   Abort transaction ('minIncrementPerKg must be ≥ 0');
22   end;
23 aKey = ctx.stub.createCompositeKey('auction', [lotId]);
24 existing = Fetch aKey from ledger;
25 nowIso = this.simNowISO(ctx);
26 if (existing?.length) {;
27   a = Parse JSON existing.toString();
28   if (a.paid —— lot.status === 'PAID') {;
29     Abort transaction ('Cannot (re)open auction for ${lotId}: payment
30       already recorded / lot is PAID');
31     end;
32   if (a.status === 'OPEN') {;
33     a.reservePricePerKg = reserve;
34     a.minIncrementPerKg = inc;
35     Update ledger with aKey, Buffer.from(Serialize a);
36     ctx.stub.setEvent('AuctionOpenIdempotent', Buffer.from(Serialize {
37       lotId, status: a.status }));
38     return 'Auction already OPEN for ${lotId} params updated';
39   end;
40   a.status = 'OPEN';
```

Algorithm 8: openAuction() (Part 2 of 2)

Output: Updated ledger state

```
1 a.timePolicy = 'MANUAL';
2 a.startTime = nowIso;
3 a.endTime = null;
4 a.reservePricePerKg = reserve;
5 a.minIncrementPerKg = inc;
6 a.bidRound = Number(a.bidRound — 1) + 1;
7 delete a.highestBid // clear previous winner/bid;
8 a.reopenedBy = this._userId ? this._userId(ctx) : 'unknown';
9 a.reopenedAt = nowIso;
10 Update ledger with aKey, Buffer.from(Serialize a);
11 ctx.stub.setEvent('AuctionReopened', Buffer.from(Serialize a));
12 return ' Auction reopened for ${lotId} (manual close)';
13 end;
14 auction = {;
15 type: 'auction',;
16 lotId,;
17 openedBy: this._userId ? this._userId(ctx) : 'unknown',;
18 status: 'OPEN',;
19 timePolicy: 'MANUAL',;
20 startTime: nowIso,;
21 endTime: null,;
22 bidRound: 1,;
23 highestBid: null,;
24 reservePricePerKg: reserve,;
25 minIncrementPerKg: inc;
26 end;
27 Update ledger with aKey, Buffer.from(Serialize auction);
28 ctx.stub.setEvent('AuctionOpened', Buffer.from(Serialize auction));
29 return ' Auction opened for ${lotId} (manual close)';
```

Algorithm 9: placeBid() (Part 1 of 2)

Input: lotId, pricePerKg, fundingSource

```
1 this.requireOrg(ctx, 'WholesalersMSP');
2 msp = ctx.clientIdentity.getMSPID();
3 bidderId = this._userId ? this._userId(ctx);
4 : (ctx.clientIdentity.getAttributeValue('hf.EnrollmentID') —
5   'unknown');
6 aKey = ctx.stub.createCompositeKey('auction', [lotId]);
7 aBytes = Fetch aKey from ledger;
8 if (!aBytes?.length) Abort transaction ('Auction not found');
9 a = Parse JSON aBytes.toString();
10 if (a.status !== 'OPEN') Abort transaction ('Auction is not OPEN');
11 ledgerNow = this._simNowISO(ctx);
12 if (a.timePolicy !== 'MANUAL' && a.endTime && ledgerNow > new
13   Date(a.endTime)) {;
14   Abort transaction ('Auction already ended');
15 end;
16 lotKey = ctx.stub.createCompositeKey('lot', [lotId]);
17 lotBytes = Fetch lotKey from ledger;
18 if (!lotBytes?.length) Abort transaction ('Lot not found');
19 lot = Parse JSON lotBytes.toString();
20 p = Number(pricePerKg);
21 if (!Number.isFinite(p) — p != 0) Abort transaction ('Invalid bid');
22 total = Number((p * Number(lot.weightKg)).toFixed(2));
23 validSources = ['SELF', 'FINANCE'];
24 src = String(fundingSource — ).toUpperCase();
25 if (!validSources.includes(src)) {;
26   Abort transaction ('Invalid funding source. Must be one of:
27     ${validSources.join(', ')}');
28 end;
29 ts = ledgerNow;
30 auctionId = this._auctionId(lotId, a.bidRound — 1);
31 offer = {;
32   type: 'auctionOffer';
33   auctionId;
34   lotId;
35   bidRound: Number(a.bidRound — 1);
36   bidderId;
37   bidderMSP: msp, // i- NEW: store MSP;
38   pricePerKg: p;
39   totalAmount: total;
40   fundingSource: src;
```

Algorithm 10: placeBid() (Part 2 of 2)

Output: Updated ledger state

```
1 ts;
2 end;
3 offerKey = ctx.stub.createCompositeKey('auctionOffer', [auctionId,
4   '${msp}:${bidderId}:${ts}']);
5 Update ledger with offerKey, Buffer.from(Serialize offer);
6 ctx.stub.setEvent('BidPlaced', Buffer.from(JSON.stringify({
7   auctionId,;
8   lotId,;
9   bidRound: Number(a.bidRound — 1),;
10  bidderId,;
11  bidderMSP: msp,;
12  pricePerKg: p,;
13  totalAmount: total,;
14  fundingSource: src,;
15  ts;
16 })));
17 return ' Bid recorded for ${lotId}: ${p}/kg (total ${total})
18   [${src}]';
```

Algorithm 11: closeAuction() (Part 1 of 2)

Input: lotId

```
1 this.requireOrg(ctx, 'AuctioncentersMSP') // still limit to
2   AuctionCenters;
3 userId = this._userId(ctx) // e.g., "User3";
4 aKey = ctx.stub.createCompositeKey('auction', [lotId]);
5 aBytes = Fetch aKey from ledger;
6 if (!aBytes?.length) Abort transaction ('Auction not found');
7 a = Parse JSON aBytes.toString();
8 if (a.status !== 'OPEN') return ' Auction already ${a.status}';
9 if (a.openedBy !== userId) {;
10 Abort transaction ('Only the opener (${a.openedBy}) can close this
11   auction');
12 end;
13 lotKey = ctx.stub.createCompositeKey('lot', [lotId]);
14 lot = Parse JSON (Fetch lotKey from ledger.toString());
15 a.status = 'CLOSED';
16 a.closedBy = userId;
17 a.closedAt = this._simNowISO(ctx);
18 auctionId = this._auctionId(lotId, a.bidRound — 1);
19 a.highestBid = this._getBestOfferForAuction(ctx, auctionId);
20 if (!a.highestBid) {;
21 Update ledger with aKey, Buffer.from(Serialize a);
22 ctx.stub.setEvent(
23   'AuctionClosed',;
24   Buffer.from(Serialize { lotId, result: 'NO_BIDS', closedBy: userId
25     });
26 );
27 return ' Auction closed by ${userId}. No bids for ${lotId}';
28 end;
29 if (a.reservePricePerKg !== null && Number(a.highestBid.pricePerKg) <
30   Number(a.reservePricePerKg)) {;
31 a.highestBid = null;
32 Update ledger with aKey, Buffer.from(Serialize a);
33 ctx.stub.setEvent(
34   'AuctionClosed',;
35   Buffer.from(Serialize { lotId, result: 'RESERVE_NOT_MET', closedBy:
36     userId });
37 );
38 return ' Auction closed by ${userId}. Reserve not met for
39   ${lotId}';
40 end;
```

Algorithm 12: closeAuction() (Part 2 of 2)

Output: Updated ledger state

```
1 lot.status = 'BID';
2 lot.soldAt = this.simNowISO(ctx) // deterministic;
3 lot.totalPrice = a.highestBid.totalAmount;
4 lot.acceptedOffer = {};
5 wholesalerId: a.highestBid.bidderId;;
6 offerPrice: a.highestBid.pricePerKg;;
7 totalAmount: a.highestBid.totalAmount;
8 end;
9 Update ledger with lotKey, Buffer.from(Serialize lot);
10 Update ledger with aKey, Buffer.from(Serialize a);
11 ctx.stub.setEvent(;
12 'AuctionClosed';
13 Buffer.from(Serialize { lotId, winner: a.highestBid, closedBy: userId
14   });
15 );
16 return ' Auction closed by ${userId}. Winner
17   ${a.highestBid.bidderId} @ ${a.highestBid.pricePerKg}/kg';
```

Algorithm 13: farmerReviewBidForLot() (Part 1 of 3)

Input: lotId, decision

```
1 this.requireOrg(ctx, 'FarmersMSP');
2 dec = String(decision —— ").toUpperCase();
3 if (![ 'ACCEPT', 'REJECT'].includes(dec)) {;
4   Abort transaction ("decision must be 'ACCEPT' or 'REJECT'");
5 end;
6 lotKey = ctx.stub.createCompositeKey('lot', [lotId]);
7 lotBytes = Fetch lotKey from ledger;
8 if (!lotBytes?.length) Abort transaction ('Lot not found');
9 lot = Parse JSON lotBytes.toString();
10 callerId = this._userId ? this._userId(ctx) :
11   ctx.clientIdentity.getID();
12 if (lot.farmerId && lot.farmerId !== callerId) {;
13   console.error('REVIEW_GUARD', { lotId, expectedFarmer: lot.farmerId,
14     caller: this._userId(ctx), msp: ctx.clientIdentity.getID() });
15   Abort transaction ('Only the farmer '${lot.farmerId}'
16     '${callerId}' can review this lot');
17 end;
18 if (lot.status !== 'BID') {;
19   Abort transaction ('Lot status must be 'BID' to review (current:
20     '${lot.status}')');
21 end;
22 aKey = ctx.stub.createCompositeKey('auction', [lotId]);
23 aBytes = Fetch aKey from ledger;
24 if (!aBytes?.length) Abort transaction ('Auction not found for this
25   lot');
26 auction = Parse JSON aBytes.toString();
27 if (auction.status !== 'CLOSED') {;
28   Abort transaction ('Auction must be 'CLOSED' to review (current:
29     '${auction.status}')');
30 end;
31 if (!auction.highestBid) Abort transaction ('No highestBid to review');
32 nowIso = this._simNowISO(ctx);
33 if (dec === 'ACCEPT') {;
34   highestBid = auction.highestBid —— {};
35   bidderId = String(highestBid.bidderId —— ");
36   fundingSource = String(highestBid.fundingSource ——
37     'SELF').toUpperCase();
38   if (fundingSource === 'SELF') {;
39     amount = Number(highestBid.totalAmount);
40     if (!bidderId) Abort transaction ('Winning bidder not found for self-
```

Algorithm 14: farmerReviewBidForLot() (Part 2 of 3)

```
1 funded settlement');
2 if (!Number.isFinite(amount) — amount != 0) {;
3   Abort transaction ('Invalid self-funded winning amount:
4     ${highestBid.totalAmount}');
5   end;
6   Transfer tokens ctx, 'wholesalers', bidderId, 'farmers', callerId,
7     String(amount);
8   lot.status = 'SOLD';
9   lot.acceptedByFarmerAt = nowIso;
10  lot.paidBy = bidderId;
11  lot.paidAt = nowIso;
12  lot.ownerId = bidderId;
13  lot.wholesalerId = bidderId;
14  auction.paid = true;
15  auction.paidTx = ctx.stub.getTxID();
16  auction.paidAt = nowIso;
17  payment = {;
18    docType: 'payment',;
19    lotId: String(lotId),;
20    amount: Number(amount.toFixed(2)),;
21    source: 'WALLET',;
22    financierId: null,;
23    from: 'wholesalers.${bidderId}',;
24    to: 'farmers.${callerId}',;
25    bidderId,;
26    farmerId: callerId,;
27    txId: ctx.stub.getTxID(),;
28    at: nowIso,;
29    status: 'SUCCESS';
30  end;
31  payKey = ctx.stub.createCompositeKey('payment', [String(lotId),
32    bidderId]);
33  Update ledger with payKey, Buffer.from(Serialize payment);
34  ctx.stub.setEvent('PaymentToFarmer', Buffer.from(Serialize payment));
35  } else {;
36  lot.status = 'BID-ACCEPTED';
37  lot.acceptedByFarmerAt = nowIso;
38  end;
39  } else { // REJECT;
40  lot.status = 'BID-REJECTED';
```

Algorithm 15: farmerReviewBidForLot() (Part 3 of 3)

Output: Updated ledger state

```
1 lot.rejectedByFarmerAt = nowIso;
2 auction.status = 'OPEN';
3 auction.reopenedByFarmer = true;
4 auction.reopenedAt = nowIso;
5 delete auction.highestBid;
6 end;
7 Update ledger with lotKey, Buffer.from(Serialize lot);
8 Update ledger with aKey, Buffer.from(Serialize auction);
9 evt = { lotId, decision: dec, lotStatus: lot.status, auctionStatus:
10   auction.status, at: nowIso };
11 ctx.stub.setEvent('FarmerReviewedBid', Buffer.from(Serialize evt));
12 return Serialize evt;
```

Algorithm 16: createFinanceRequestForLot() (Part 1 of 3)

Input: lotId, financierId, tenorDaysRequested

```
1 this.requireOrg(ctx, 'WholesalersMSP');
2 if (!lotId) Abort transaction ('lotId required');
3 if (!financierId) Abort transaction ('financierId required');
4 requesterId = this._userId ? this._userId(ctx) : 'unknown';
5 lotKey = ctx.stub.createCompositeKey('lot', [String(lotId)]);
6 lotBytes = Fetch lotKey from ledger;
7 if (!lotBytes?.length) Abort transaction ('Lot ${lotId} not found');
8 lot = Parse JSON lotBytes.toString();
9 aKey = ctx.stub.createCompositeKey('auction', [String(lotId)]);
10 aBytes = Fetch aKey from ledger;
11 if (!aBytes?.length) Abort transaction ('Auction not found for lot
12   ${lotId}');
13 auction = Parse JSON aBytes.toString();
14 if (lot.status !== 'BID-ACCEPTED') {
15   Abort transaction ('Lot ${lotId} not approved by farmer
16     (status=${lot.status})');
17   end;
18   if (auction.status !== 'CLOSED') {
19     Abort transaction ('Auction for lot ${lotId} must be CLOSED
20       (current=${auction.status})');
21     end;
22     if (!auction.highestBid) Abort transaction ('No highest bid recorded');
23     { bidderId, totalAmount } = auction.highestBid;
24     if (String(bidderId) !== String(requesterId)) {
25       Abort transaction ('Only the winning bidder (${bidderId}) can
26         request finance for this lot');
27     }
28     principal = Number(totalAmount);
29     if (!Number.isFinite(principal) ——— principal != 0) {
30       Abort transaction ('Invalid principal from auction:
31         ${totalAmount}');
32     }
33     reqId = ctx.stub.getTxID();
34     policy = this._getFinancePolicyFor(ctx, financierId, 'false');
35     maxTenorDays = Number(policy?.tenorDays ?? 0);
36     feePct = Number(policy?.feePct ?? 0);
37     penaltyPct = Number(policy?.penaltyPct ?? 0);
38     tenorDays = maxTenorDays < 0 ? maxTenorDays : 30;
39     if (tenorDaysRequested !== undefined && tenorDaysRequested !== null
40       && String(tenorDaysRequested).trim() !== "") {;
```

Algorithm 17: createFinanceRequestForLot() (Part 2 of 3)

```
1 t = Number(tenorDaysRequested);
2 if (!Number.isFinite(t) — t != 0) Abort transaction
3   ('tenorDaysRequested must be a positive number');
4 tenorDays = maxTenorDays > 0 ? Math.min(t, maxTenorDays) : t;
5 end;
6 if (!Number.isFinite(tenorDays) — tenorDays != 0) {;
7   Abort transaction ('A valid tenorDays is required to create the
8     finance request');
9 end;
10 createdIso = this._simNowISO ? this._simNowISO(ctx) : new
11   Date().toISOString();
12 dueIso = new Date(Date.parse(createdIso) + tenorDays *
13   86400000).toISOString();
14 reqKey = ctx.stub.createCompositeKey('financeReq', [String(lotId),
15   String(requesterId)]);
16 prev = Fetch reqKey from ledger;
17 if (prev?.length) {;
18   p = Parse JSON prev.toString();
19   if (p.status !== 'REJECTED') {;
20     Abort transaction ('A finance request already exists for lot
21       ${lotId} by ${requesterId} (status=${p.status})');
22   end;
23 end;
24 request = {;
25   docType: 'financeRequest';
26   requestId: reqId;
27   lotId;
28   financierId;
29   requesterId;
30   requesterOrg: 'WholesalersMSP';
31   principalAmount: Number(principal.toFixed(2));
32   outstanding: Number(principal.toFixed(2));
33   repaidAmt: 0;
34   wholesalePaid: 0;
35   penaltyAccum: 0;
36   overdue: 0;
37   payments: [];
38   pricing: {;
39     feePct;
40     annualInterestPct: 0;
```

Algorithm 18: createFinanceRequestForLot() (Part 3 of 3)

Output: Updated ledger state

```
1 aprEnteredByFinancier: false;;
2 penaltyPct;;
3 tenorDays;;
4 snapshotTs: createdIso;
5 },;
6 computed: {};
7 feeAmt: 0;;
8 interestAmt: 0;;
9 totalPayable: Number(principal.toFixed(2));
10 },;
11 status: 'PENDING';
12 createdAt: createdIso;;
13 dueAt: dueIso;;
14 termsSnapshot: {};
15 source: 'FINANCIER_REVIEW_PENDING';
16 score: null;;
17 maxTenorDays: maxTenorDays < 0 ? maxTenorDays : null;
18 end;
19 end;
20 Update ledger with reqKey, Buffer.from(Serialize request);
21 ctx.stub.setEvent('FinanceRequested', Buffer.from(JSON.stringify({
22   lotId;;
23   requestId: request.requestId;;
24   requesterId;;
25   financierId;;
26   principal: request.principalAmount;;
27   tenorDays;;
28   feePct;;
29   aprPct: null;;
30   penaltyPct;;
31   source: 'FINANCIER_REVIEW_PENDING';
32 })))));
33 return 'Finance request ${request.requestId} created for lot
34   ${lotId} -> principal ${request.principalAmount}, tenor
35   ${tenorDays}d';
```

Algorithm 19: reviewFinanceRequest() (Part 1 of 3)

```
Input: requestId, approve, aprPctInput
1 this.requireOrg(ctx, 'FinanciersMSP');
2 key = ctx.stub.createCompositeKey('financeReq', [String(requestId)]);
3 bytes = Fetch key from ledger;
4 if (!bytes?.length) {;
5   iter = ctx.stub.getStateByPartialCompositeKey('financeReq', []);
6   rows = this._drainIteratorKV(iter) // i- add this.;
7   for (const { key: k, value } of rows) {;
8     try {;
9       rec = Parse JSON value;
10      if (rec?.requestId === requestId) { key = k bytes =
11        Buffer.from(Serialize rec) break };
12    } catch {};
13  end;
14  end;
15  if (!bytes?.length) Abort transaction ('Finance request not found for
16    requestId ${requestId}');
17  req = Parse JSON bytes.toString();
18  if (req.status !== 'PENDING') Abort transaction ('Request
19    ${requestId} is not PENDING (current: ${req.status})');
20  callerId = this._clientId ? this._clientId(ctx) : 'unknown';
21  if (String(req.financierId) !== String(callerId)) {;
22    Abort transaction ('Only financier ${req.financierId} can review
23      this request');
24  end;
25  decision = String(approve).toLowerCase() === 'true';
26  hasAprInput = aprPctInput !== undefined && aprPctInput !== null &&
27    String(aprPctInput).trim() !== '';
28  if (decision && hasAprInput) {;
29    rawApr = Number(aprPctInput);
30    if (!Number.isFinite(rawApr) — rawApr i= 0) {;
31      Abort transaction ('APR must be a positive number');
32    end;
33    aprPct = rawApr i= 1 ? rawApr * 100 : rawApr;
34    if (!Number.isFinite(aprPct) — aprPct i= 0 — aprPct i 100) {;
35      Abort transaction ('APR must be between 0 and 100 percent');
36    end;
37    principal = Number(req.principalAmount — 0);
38    tenorDays = Number(req?.pricing?.tenorDays — 0);
39    feePct = Number(req?.pricing?.feePct — 0);
40    if (!Number.isFinite(principal) — principal i= 0) Abort transaction
```

Algorithm 20: reviewFinanceRequest() (Part 2 of 3)

```
1  ('Finance request has invalid principalAmount');
2  if (!Number.isFinite(tenorDays) — tenorDays != 0) Abort transaction
3  ('Finance request has invalid tenorDays');
4  roundMoney = Number.isFinite(Number(req?.pricing?.roundMoney)) ?
5    Number(req.pricing.roundMoney) : 2;
6  feeAmt = this._round(principal * (feePct / 100), roundMoney);
7  interestAmt = this._round(principal * (aprPct / 100) * (tenorDays /
8    365), roundMoney);
9  totalPayable = this._round(principal + feeAmt + interestAmt,
10    roundMoney);
11 previousAprPct = Number(req?.pricing?.annualInterestPct);
12 req.pricing = {;
13 ...(req.pricing — {}),;
14 annualInterestPct: Number(aprPct.toFixed(2)),;
15 aprEnteredByFinancier: true;;
16 aprEnteredAt: new Date().toISOString();
17 end;
18 req.computed = {;
19 ...(req.computed — {}),;
20 feeAmt;;
21 interestAmt;;
22 totalPayable;
23 end;
24 req.outstanding = totalPayable;
25 req.termsSnapshot = {;
26 ...(req.termsSnapshot — {}),;
27 aprSource: 'FINANCIER_INPUT';
28 previousAprPct: Number.isFinite(previousAprPct) ? previousAprPct :
29   null;
30 end;
31 end;
32 req.status = decision ? 'APPROVED' : 'REJECTED';
33 req.reviewedAt = new Date().toISOString();
34 req.reviewerId = callerId;
35 delete req.reviewerNote;
36 Update ledger with key, Buffer.from(Serialize req);
37 ctx.stub.setEvent('FinanceRequestReviewed', Buffer.from(Serialize {
38   requestId: req.requestId, status: req.status }));
39 if (decision) {;
40 return this._disburseApprovedFinanceForLot(ctx, req.lotId,
```

Algorithm 21: reviewFinanceRequest() (Part 3 of 3)

Output: Updated ledger state

```
1 req.requesterId, req);  
2 end;  
3 return ‘ Request ${req.requestId} ${req.status}’;
```

Algorithm 22: approveFinanceRequestWithAPR()

Input: requestId, aprPct, note=**Output:** Updated ledger state

```
1 return this.reviewFinanceRequest(ctx, requestId, ‘true’, note, aprPct);
```

Algorithm 23: updateWholesalerScoreOnSettlement() (Part 1 of 2)

Input: wholesalerId, status, tenorDays, daysLate

```
1 async listFinancierOffersForWholesaler(ctx, principalAmount = "",
2   tenorDaysRequested = "") {};
3 this._requireOrg(ctx, 'WholesalersMSP');
4 wholesalerId = this._userId ? this._userId(ctx) : 'unknown';
5 principal = Number(principalAmount — 0);
6 requestedTenor = Number(tenorDaysRequested — 0);
7 includePricing = Number.isFinite(principal) && principal > 0;
8 iter = ctx.stub.getStateByPartialCompositeKey('finOffer',
9   [String(wholesalerId)]);
10 offers = [];
11 seen = new Set();
12 for (let r = iter.next() !r.done r = iter.next()) {};
13 if (!r.value?.value) continue;
14 offer = Parse JSON r.value.value.toString('utf8');
15 seen.add(String(offer.financierId));
16 tenorDays = Number.isFinite(requestedTenor) && requestedTenor > 0;
17 ? Math.min(requestedTenor, Number(offer.tenorDays — requestedTenor));
18 : Number(offer.tenorDays — 0);
19 row = {};
20 financierId: offer.financierId;
21 wholesalerId;
22 source: offer.source — 'RL';
23 aprPct: Number(offer.aprPct);
24 baseAprPct: Number(offer.baseAprPct);
25 feePct: Number(offer.feePct ?? 1.0);
26 penaltyPct: Number(offer.penaltyPct ?? 0);
27 tenorDays;
28 ltvCapPct: Number(offer.ltvCapPct ?? 0);
29 updatedAt: offer.updatedAt;
30 rl: offer.rl — null;
31 end;
32 if (includePricing && Number.isFinite(row.aprPct) &&
33   Number.isFinite(tenorDays) && tenorDays > 0) {};
34 feeAmt = this._round(principal * (row.feePct / 100), 2);
35 interestAmt = this._round(principal * (row.aprPct / 100) * (tenorDays
36   / 365), 2);
37 row.pricing = {};
38 principalAmount: principal;
39 feeAmt;
40 interestAmt;
```

Algorithm 24: updateWholesalerScoreOnSettlement() (Part 2 of 2)

Output: Updated ledger state

```
1 totalPayable: this._round(principal + feeAmt + interestAmt, 2);
2 end;
3 end;
4 offers.push(row);
5 end;
6 iter.close();
7 financierIds = this._listKnownFinancierIds(ctx);
8 for (const financierId of financierIds) {;
9   if (seen.has(String(financierId))) continue;
10  offer = this._buildAutoRLOffer(ctx, wholesalerId, financierId,
11    principalAmount, tenorDaysRequested);
12  offers.push({;
13    financierId: offer.financierId;;
14    wholesalerId;;
15    source: offer.source;;
16    aprPct: Number(offer.aprPct);;
17    baseAprPct: Number(offer.baseAprPct);;
18    feePct: Number(offer.feePct ?? 1.0);;
19    penaltyPct: Number(offer.penaltyPct ?? 0);;
20    tenorDays: Number(offer.tenorDays);;
21    ltvCapPct: Number(offer.ltvCapPct ?? 0);;
22    updatedAt: offer.updatedAt;;
23    rl: offer.rl — null;;
24    pricing: offer.pricing — undefined;
25  });
26 end;
27 offers.sort((a, b) => Number(a.aprPct) - Number(b.aprPct));
28 return Serialize offers;
```

Algorithm 25: packLotIntoPackets() (Part 1 of 4)

Input: lotId, sizesJson, percentagesJson, pricesJson, packingVideoHash

```
1 this.requireOrg(ctx, 'WholesalersMSP');
2 lotKey = ctx.stub.createCompositeKey('lot', [lotId]);
3 lotBytes = Fetch lotKey from ledger;
4 if (!lotBytes ——— lotBytes.length === 0) Abort transaction ('Lot not
5   found');
6 lot = Parse JSON lotBytes.toString();
7 if (lot.status === 'PACKED') {;
8   Abort transaction ('Lot ${lotId} already packed');
9   end;
10 if (lot.status !== 'SOLD') {;
11   Abort transaction ('Lot must be APPROVED or SOLD to be packed');
12   end;
13 sizes, percentages, prices;
14 try {;
15   sizes = Parse JSON sizesJson // array of numbers (grams);
16   percentages = Parse JSON percentagesJson // array OR object;
17   prices = Parse JSON pricesJson // object { "1000": 1200, ...
18     end;
19 } catch (e) {;
20   Abort transaction ('Invalid JSON for sizes/percentages/prices:
21     ${e.message}');
22   end;
23 if (!Array.isArray(sizes) ——— sizes.length === 0) {;
24   Abort transaction ("sizesJson must be a non-empty JSON array of packet
25     sizes in grams");
26   end;
27 if (typeof prices !== 'object' ——— prices === null) {;
28   Abort transaction ("pricesJson must be a JSON object mapping
29     size->price");
30   end;
31 sizes = [...new Set(sizes.map(n => parseInt(n, 10)))]
32   .filter(n => n > 0)
33   .sort((a,b)=>b-a);
34 if (sizes.length === 0) Abort transaction ("No valid packet sizes
35   provided");
35 pctBySize = {};
36 if (Array.isArray(percentages)) {;
37   if (percentages.length !== sizes.length) {;
38     Abort transaction ("If percentagesJson is an array, it must be the
39       same length as sizesJson");
40   }
41   end;
```

Algorithm 26: packLotIntoPackets() (Part 2 of 4)

```
1 sizes.forEach((sz, i) => {;
2   v = Number(percentages[i]);
3   if (!(v >= 0)) Abort transaction ("Percentages must be non-negative
4     numbers");
5   pctBySize[String(sz)] = v;
6 });
7 } else if (typeof percentages === 'object' && percentages !== null)
8   {;
9   sizes.forEach(sz => {;
10    v = Number(percentages[String(sz)]);
11    if (!(v >= 0)) Abort transaction ('Missing/non-numeric percentage for
12      size ${sz}');
13    pctBySize[String(sz)] = v;
14  });
15  } else {;
16    Abort transaction ("percentagesJson must be a JSON array or an object
17      keyed by size");
18  end;
19  pctSum = Object.values(pctBySize).reduce((a,b)=>a+Number(b),0);
20  if (Math.abs(pctSum - 100) > 1e-6) {;
21    Abort transaction ('Percentages must sum to 100. Got ${pctSum}');
22  end;
23  sizes.forEach(sz => {;
24    p = prices[String(sz)];
25    if (!(Number(p) >= 0)) Abort transaction ('Missing/invalid price for
26      size ${sz}g');
27  });
28  totalWeightGrams = Math.floor((lot.weightKg — 0) * 1000);
29  if (!(totalWeightGrams >= 0)) Abort transaction ("Lot weightKg must be
30    > 0 to pack");
31  countsBySize = {};
32  usedGrams = 0;
33  sizes.forEach(sz => {;
34    gramsForThisSize = Math.floor(totalWeightGrams *
35      (pctBySize[String(sz)] / 100));
36    count = Math.floor(gramsForThisSize / sz);
37    countsBySize[String(sz)] = count;
38    usedGrams += count * sz;
39  });
40  remainder = totalWeightGrams - usedGrams;
```

Algorithm 27: packLotIntoPackets() (Part 3 of 4)

```
1 smallest = sizes[sizes.length - 1];
2 while (remainder >= smallest) {;
3   countsBySize[String(smallest)] += 1;
4   remainder -= smallest;
5 } end;
6 counter = 1;
7 packetCounts = {};
8 now = new Date().toISOString();
9 putOps = [];
10 for (const sz of sizes) {;
11   sizeKey = String(sz);
12   packetCounts['${sz}g'] = 0;
13   howMany = countsBySize[sizeKey] — 0;
14   for (let i = 0 ; i < howMany ; i++) {;
15     packetId = `${lotId}-PKT-${counter}`;
16     packetKey = ctx.stub.createCompositeKey('packet', [packetId]);
17     packet = {;
18       packetId: packetId;
19       lotId: lotId;
20       weight: `${sz}g`;
21       wholesalePrice: parseFloat(prices[sizeKey]);
22       retailPrice: parseFloat(prices[sizeKey]);
23       salePrice: parseFloat(prices[sizeKey]);
24       qrCode: packetId;
25       lotRef: lotId;
26       status: 'AVAILABLE_FOR_PURCHASE';
27       packedAt: now;
28       priceSetAt: now;
29       priceSource: 'PACK_TIME';
30       videoHash: {;
31         testing: lot.videoHash — null;
32         packing: packingVideoHash;
33       };
34       trace: {;
35         farmerId: lot.farmerId;
36         submittedAt: lot.submittedAt — null;
37         aggregatorId: lot.aggregatorId — null;
38         wholesalerId: lot.wholesalerId — null;
39         testedBy: lot.aggregatorId ? `aggregators.${lot.aggregatorId}` :
40           null;
```

Algorithm 28: packLotIntoPackets() (Part 4 of 4)

Output: Updated ledger state

```
1 testedAt: lot.testedAt —— null;;
2 testingVideoHash: lot.videoHash —— null;;
3 testResult: lot.testResult —— null;;
4 packedBy: this._clientId(ctx);
5 packedAt: now;;
6 packingVideoHash;;
7 offeredDirectlyByWholesaler: true;
8 end;
9 end;
10 this._setPacketOwner(packet, 'wholesalers', this._clientId(ctx));
11 putOps.push(ctx.stub.putState(packetKey, Buffer.from(Serialize
12   packet)));
13 counter++;
14 packetCounts['${sz}g']++;
15 end;
16 end;
17 lot.packetCounts = packetCounts;
18 lot.status = 'PACKED';
19 lot.packingConfig = {};
20 sizes: sizes.map(s => '${s}g');
21 percentages: pctBySize, // keyed by grams as string;
22 prices, // keyed by grams as string;
23 totalWeightGrams: totalWeightGrams;
24 allocatedGrams: totalWeightGrams - remainder;
25 remainderGrams: remainder;
26 end;
27 lot.packetPricing = {};
28 prices;;
29 source: 'PACK_TIME';
30 setAt: now;
31 end;
32 putOps.push(ctx.stub.putState(lotKey, Buffer.from(Serialize lot)));
33 Promise.all(putOps);
34 return 'Packed ${lotId} into ${counter - 1} packets. Remainder:
35   ${remainder} g.';
```

Algorithm 29: setPacketPriceForWeight() (Part 1 of 2)

Input: weightStr, price

```
1 this.requireOrg(ctx, 'WholesalersMSP');
2 wholesalerId = this._clientId(ctx);
3 wRaw = String(weightStr ?? "").trim();
4 if (!wRaw) Abort transaction ('weightStr is required (e.g., "500g")');
5 targetWeight = /g$/i.test(wRaw) ? wRaw : `${wRaw}g`;
6 p = Number(price);
7 if (!Number.isFinite(p) — p < 0) {
8   Abort transaction ('price must be a non-negative number (got
9     `${price}`)');
10 end;
11 updated = 0;
12 iter = ctx.stub.getStateByPartialCompositeKey('packet', []);
13 try {
14   res = iter.next();
15   while (!res.done) {
16     { key, value } = res.value — { };
17     if (value && value.length) {
18       packet = Parse JSON value.toString();
19       cur = String(packet.status).toUpperCase();
20       owner = this._getPacketOwner(packet);
21       if (owner?.org === 'wholesalers' &&
22         String(owner?.id) === wholesalerId &&
23         String(packet.weight).trim() === targetWeight &&
24         cur === 'AVAILABLE_FOR_PURCHASE'){
25         packet.wholesalePrice = p;
26         if (!Number.isFinite(Number(packet.retailPrice)) —
27           Number(packet.retailPrice) <= 0) {
28           packet.retailPrice = p;
29         end;
30         packet.status = 'AVAILABLE_FOR_PURCHASE';
31         Update ledger with key, Buffer.from(Serialize packet);
32         updated++;
33       end;
34     end;
35   } finally {
36     res = iter.next();
37   }
38   if (iter?.close) iter.close();
39 end;
40 payload = {
```

Algorithm 30: setPacketPriceForWeight() (Part 2 of 2)

Output: Updated ledger state

```
1 event: 'PacketPricesUpdated',;  
2 wholesalerId,;  
3 weight: targetWeight,;  
4 wholesalePrice: p,;  
5 updated,;  
6 fromStatus: 'AVAILABLE_FOR_PURCHASE',;  
7 newStatus: 'AVAILABLE_FOR_PURCHASE',;  
8 end;  
9 ctx.stub.setEvent('PacketPricesUpdated', Buffer.from(Serialize  
10   payload));  
11 return Serialize payload;
```

Algorithm 31: setRetailerPriceForWeight() (Part 1 of 2)

Input: weightStr, price

```
1 this.requireOrg(ctx, 'RetailersMSP');
2 retailerId = this._clientId(ctx);
3 wRaw = String(weightStr ?? "").trim();
4 if (!wRaw) Abort transaction ('weightStr is required (e.g., "500g")');
5 targetWeight = /g$/i.test(wRaw) ? wRaw : `${wRaw}g`;
6 p = Number(price);
7 if (!Number.isFinite(p) — p < 0) {
8   Abort transaction ('price must be a non-negative number (got
9     `${price}`)');
10 end;
11 updated = 0;
12 iter = ctx.stub.getStateByPartialCompositeKey('packet', []);
13 try {
14   res = iter.next();
15   while (!res.done) {
16     { key, value } = res.value — { };
17     if (value && value.length) {
18       packet = Parse JSON value.toString();
19       owner = this._getPacketOwner(packet);
20       cur = String(packet.status — ).toUpperCase();
21       if (owner?.org === 'retailers' &&
22         String(owner?.id) === retailerId &&
23         String(packet.weight).trim() === targetWeight &&
24         ['AVAILABLE_FOR_CONSUMER_PURCHASE',
25           'OWNED_BY_RETAILER'].includes(cur)) {
26         packet.retailPrice = p;
27         packet.salePrice = p;
28         packet.status = 'AVAILABLE_FOR_CONSUMER_PURCHASE';
29         Update ledger with key, Buffer.from(Serialize packet);
30         updated++;
31       end;
32     end;
33     res = iter.next();
34   end;
35 } finally {
36   if (iter?.close) iter.close();
37 end;
38 payload = {
39   event: 'RetailerPricesUpdated',;
40   retailerId,;
```

Algorithm 32: setRetailerPriceForWeight() (Part 2 of 2)

Output: Updated ledger state

```
1 weight: targetWeight;;
2 retailPrice: p;;
3 updated;;
4 newStatus: 'AVAILABLE_FOR_CONSUMER_PURCHASE';
5 end;
6 ctx.stub.setEvent('RetailerPricesUpdated', Buffer.from(Serialize
7   payload));
8 return Serialize payload;
```

Algorithm 33: purchasePacket() (Part 1 of 3)

Input: packetId, consumerId

```
1 this.requireOrg(ctx, 'ConsumersMSP');
2 nowIso = this.simNowISO(ctx);
3 packetKey = ctx.stub.createCompositeKey('packet', [String(packetId)]);
4 packetBytes = Fetch packetKey from ledger;
5 if (!packetBytes?.length) Abort transaction ('Packet ${packetId}
6   not found');
7 packet = Parse JSON packetBytes.toString();
8 statusNow = String(packet.status ——— ").toUpperCase();
9 if (!['AVAILABLE_FOR_CONSUMER_PURCHASE',
10  'OWNED_BY_RETAILER'].includes(statusNow)) {;
11 Abort transaction ('Packet ${packetId} is not
12  AVAILABLE_FOR_CONSUMER_PURCHASE (current: ${packet.status}');
13 end;
14 owner = this.getPacketOwner ? this.getPacketOwner(packet) : {
15   org: 'retailers', id: packet.owner };
16 if (String(owner.org).toLowerCase() !== 'retailers') {;
17 Abort transaction ('Packet ${packetId} is not owned by a retailer
18   (ownerOrg=${owner.org}');
19 end;
20 retailerId = owner.id;
21 consumerPrice = Number(packet.retailPrice ?? packet.salePrice ??
22   packet.wholesalePrice);
23 if (!Number.isFinite(consumerPrice) ——— consumerPrice != 0) {;
24 Abort transaction ('Invalid packet retail price:
25   ${packet.retailPrice ?? packet.salePrice ??
26   packet.wholesalePrice}');
27 end;
28 wholesalePaid = 0;
29 lotId = String(packet.lotRef ——— packet.lotId ——— ").trim();
30 wholesalerId = null;
31 if (lotId) {;
32 lotKey = ctx.stub.createCompositeKey('lot', [lotId]);
33 lotBytes = Fetch lotKey from ledger;
34 if (lotBytes?.length) {;
35 lot = Parse JSON lotBytes.toString();
36 wholesalerId = lot.ownerId ——— lot?.acceptedOffer?.wholesalerId ——— null;
37 end;
38 end;
39 Transfer tokens ctx, 'consumers', '${consumerId}', 'retailers',
40   '${retailerId}', String(consumerPrice);
```

Algorithm 34: purchasePacket() (Part 2 of 3)

```
1 financeRepay = 0;
2 if (this._setPacketOwner) this._setPacketOwner(packet, 'consumers',
3   consumerId);
4 else packet.owner = consumerId;
5 packet.status = 'PURCHASED';
6 packet.soldAt = nowIso;
7 packet.consumerId = String(consumerId);
8 packet.retailerId = String(retailerId);
9 packet.trace = packet.trace — { };
10 packet.trace.consumerId = String(consumerId);
11 packet.trace.purchasedBy = consumerId;
12 packet.trace.purchasedAt = nowIso;
13 packet.trace.consumerPurchase = { };
14 retailerId: retailerId — null;;
15 consumerPrice;
16 end;
17 packet.trace.settlement = { };
18 wholesalerId: wholesalerId — null;;
19 retailerId: retailerId — null;;
20 financeRepay;;
21 wholesalePaid;;
22 consumerPrice;
23 end;
24 Update ledger with packetKey, Buffer.from(Serialize packet);
25 ctx.stub.setEvent('PacketSold', Buffer.from(JSON.stringify({;
26 packetId;;
27 consumerId;;
28 lotId;;
29 retailerId;;
30 wholesalerId;;
31 consumerPrice;;
32 wholesalePaid;;
33 financeRepay;;
34 at: nowIso;
35 })));;
36 return JSON.stringify({;
37 ok: true;;
38 packetId;;
39 consumerId;;
40 retailerId;;
```

Algorithm 35: purchasePacket() (Part 3 of 3)

Output: Updated ledger state

```
1 wholesalerId;;  
2 lotId;;  
3 consumerPrice;;  
4 wholesalePaid;;  
5 financeRepay;;  
6 at: nowIso;  
7 });
```
